



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA N° 02

NOMBRE COMPLETO: Del Toro Ortiz Juan Pablo Yasbeth

N° de Cuenta: 317031814

GRUPO DE LABORATORIO: 13

GRUPO DE TEORÍA: 7

SEMESTRE 2026-2

FECHA DE ENTREGA LÍMITE: 26/02/2026

CALIFICACIÓN: _____

REPORTE DE PRÁCTICA:

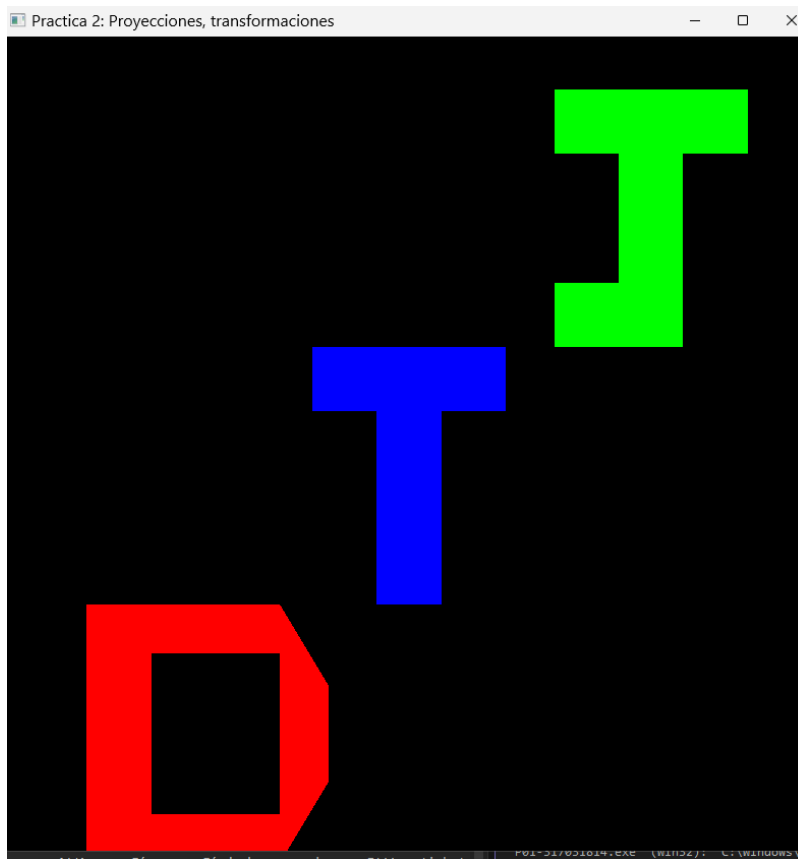
1.- Ejecución de los ejercicios que se dejaron, comentar cada uno y capturas de pantalla de bloques de código generados y de ejecución del programa.

Instrucciones:

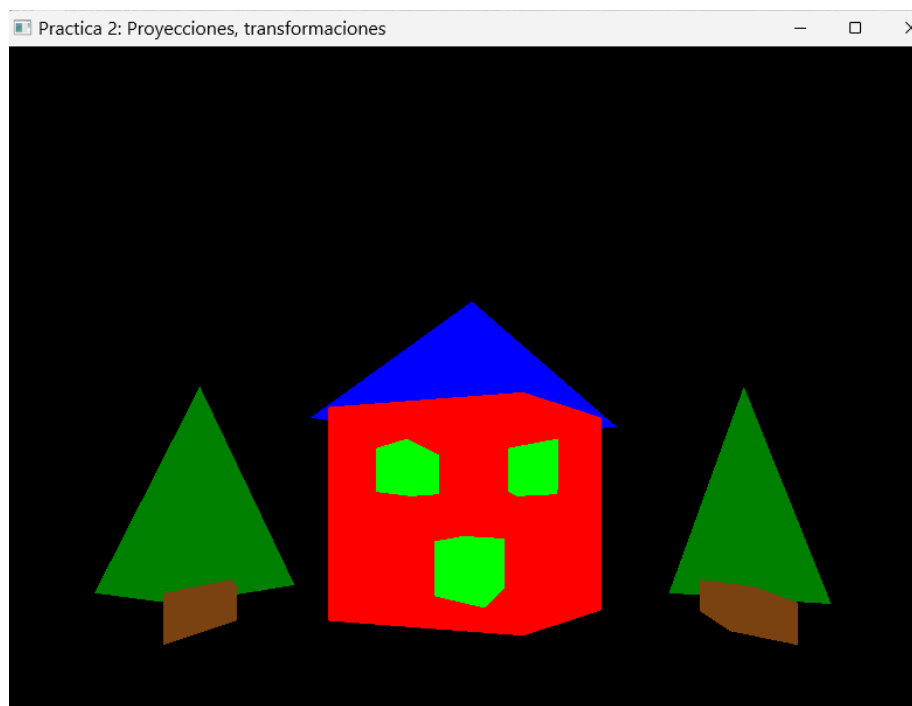
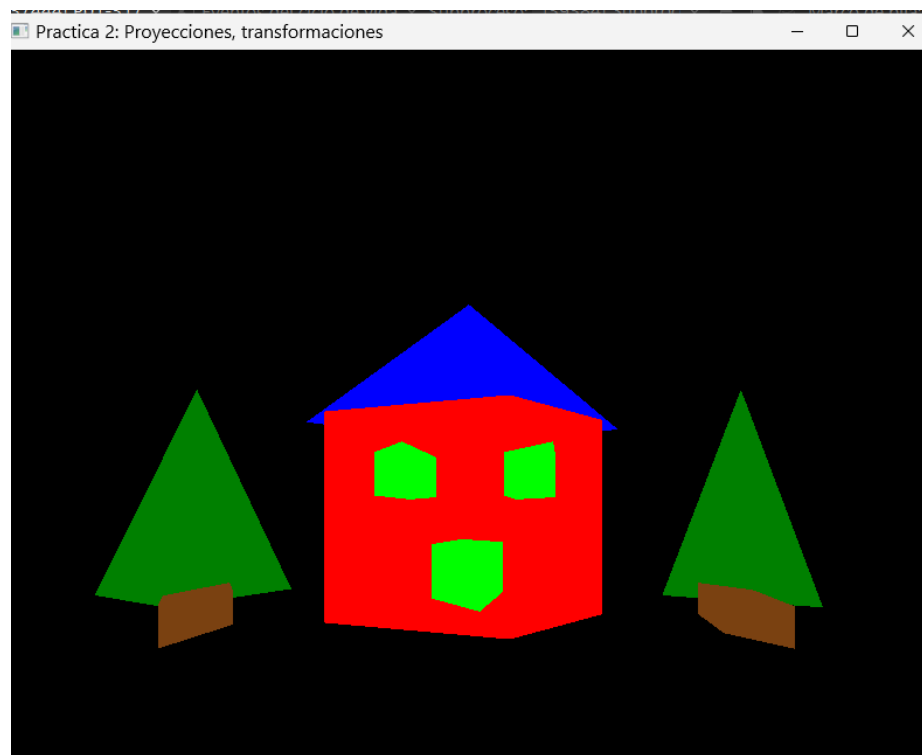
1.- Dibujar las iniciales de sus nombres en diagonal de abajo hacia arriba, cada letra de un color diferente

2.- Generar el dibujo de la casa de la clase, pero en lugar de instanciar triángulos y cuadrados será instanciando pirámides y cubos, para esto se requiere crear shaders diferentes de los colores: rojo, verde, azul, café y verde oscuro en lugar de usar el shader con el color clamp. Se debe de entregar los archivos de shader diferentes dentro de un zip adicional al main y documento escrito.

Capturas de pantalla del funcionamiento Actividad 1:



Capturas de pantalla del funcionamiento Actividad 2:



(Capturas de pantalla con intento de mostrar la rotación que tiene la figura)

Código generado:

Modificación a la figura de pirámide para su manejo como pirámide cuadrangular:

```
30 //Modificación a Pirámide (base cuadrada, 4 caras triangulares)
31 void CreaPiramide()
32 {
33     unsigned int indices[] = {
34         // Cara frontal
35         0, 1, 4,
36         // Cara derecha
37         1, 2, 4,
38         // Cara trasera
39         2, 3, 4,
40         // Cara izquierda
41         3, 0, 4,
42         // Base (cuadrada) - dividida en 2 triángulos
43         0, 2, 1,
44         0, 3, 2
45     };
46     GLfloat vertices[] = {
47         // Base - Vértices de la base cuadrada (y = -0.5f)
48         -0.5f, -0.5f, 0.5f, // 0: Frontal izquierdo
49         0.5f, -0.5f, 0.5f, // 1: Frontal derecho
50         0.5f, -0.5f, -0.5f, // 2: Trasero derecho
51         -0.5f, -0.5f, -0.5f, // 3: Trasero izquierdo
52
53         // Punta de la pirámide (y = 0.5f)
54         0.0f, 0.5f, 0.0f // 4: Punta superior
55     };
56     Mesh* obj1 = new Mesh();
57     obj1->CreateMesh(vertices, indices, 15, 18); // 5 vértices * 3 coordenadas = 15, 18 índices
58     meshList.push_back(obj1);
59 }
60
```

Función para crear la letra D:

```
81 // Función para crear la letra D mayúscula
82 void CrearLetraD()
83 {
84     // Vértices para la letra D mayúscula (estilo bloque)
85     GLfloat vertices_D[] = {
86         // Barra vertical izquierda
87         -0.8f, -0.8f, 0.5f, 1.0f, 0.0f, 0.0f, // 0
88         -0.4f, -0.8f, 0.5f, 1.0f, 0.0f, 0.0f, // 1
89         -0.4f, 0.8f, 0.5f, 1.0f, 0.0f, 0.0f, // 2
90         -0.8f, -0.8f, 0.5f, 1.0f, 0.0f, 0.0f, // 3
91         -0.4f, 0.8f, 0.5f, 1.0f, 0.0f, 0.0f, // 4
92         -0.8f, 0.8f, 0.5f, 1.0f, 0.0f, 0.0f, // 5
93         // Barra superior
94         -0.4f, 0.5f, 0.5f, 1.0f, 0.0f, 0.0f, // 6
95         0.4f, 0.5f, 0.5f, 1.0f, 0.0f, 0.0f, // 7
96         0.4f, 0.8f, 0.5f, 1.0f, 0.0f, 0.0f, // 8
97         -0.4f, 0.5f, 0.5f, 1.0f, 0.0f, 0.0f, // 9
98         0.4f, 0.8f, 0.5f, 1.0f, 0.0f, 0.0f, // 10
99         -0.4f, 0.8f, 0.5f, 1.0f, 0.0f, 0.0f, // 11
100
```

```

100      // Barra inferior
101      -0.4f, -0.8f, 0.5f,      1.0f, 0.0f, 0.0f, // 12
102      0.4f, -0.8f, 0.5f,      1.0f, 0.0f, 0.0f, // 13
103      0.4f, -0.5f, 0.5f,      1.0f, 0.0f, 0.0f, // 14
104      -0.4f, -0.8f, 0.5f,      1.0f, 0.0f, 0.0f, // 15
105      0.4f, -0.5f, 0.5f,      1.0f, 0.0f, 0.0f, // 16
106      -0.4f, -0.5f, 0.5f,      1.0f, 0.0f, 0.0f, // 17
107      // Barra vertical derecha (curva simulada)
108      0.4f, -0.8f, 0.5f,      1.0f, 0.0f, 0.0f, // 18
109      0.7f, -0.3f, 0.5f,      1.0f, 0.0f, 0.0f, // 19
110      0.7f, 0.3f, 0.5f,      1.0f, 0.0f, 0.0f, // 20
111      0.4f, -0.8f, 0.5f,      1.0f, 0.0f, 0.0f, // 21
112      0.7f, 0.3f, 0.5f,      1.0f, 0.0f, 0.0f, // 22
113      0.4f, 0.8f, 0.5f,      1.0f, 0.0f, 0.0f, // 23
114  };
115  MeshColor* letraD = new MeshColor();
116  letraD->CreateMeshColor(vertices_D, 144); // 24 vértices * 6 componentes (X,Y,Z,R,G,B)
117  meshColorList.push_back(letraD);
118  }

```

Función para crear la letra T:

```

120      // Función para crear la letra T mayúscula (azul)
121      void CrearLetraT()
122      {
123          // Vértices para la letra T mayúscula
124          GLfloat vertices_T[] = {
125              // Barra superior horizontal
126              -0.6f, 0.4f, 0.5f,      0.0f, 0.0f, 1.0f, // 0 - Azul
127              0.6f, 0.4f, 0.5f,      0.0f, 0.0f, 1.0f, // 1
128              0.6f, 0.8f, 0.5f,      0.0f, 0.0f, 1.0f, // 2
129              -0.6f, 0.4f, 0.5f,      0.0f, 0.0f, 1.0f, // 3
130              0.6f, 0.8f, 0.5f,      0.0f, 0.0f, 1.0f, // 4
131              -0.6f, 0.8f, 0.5f,      0.0f, 0.0f, 1.0f, // 5
132              // Barra vertical central
133              -0.2f, -0.8f, 0.5f,      0.0f, 0.0f, 1.0f, // 6
134              0.2f, -0.8f, 0.5f,      0.0f, 0.0f, 1.0f, // 7
135              0.2f, 0.4f, 0.5f,      0.0f, 0.0f, 1.0f, // 8
136              -0.2f, -0.8f, 0.5f,      0.0f, 0.0f, 1.0f, // 9
137              0.2f, 0.4f, 0.5f,      0.0f, 0.0f, 1.0f, // 10
138              -0.2f, 0.4f, 0.5f,      0.0f, 0.0f, 1.0f, // 11
139          };
140          MeshColor* letraT = new MeshColor();
141          letraT->CreateMeshColor(vertices_T, 72); // 12 vértices * 6 componentes
142          meshColorList.push_back(letraT);
143      }

```

Función para crear la letra J:

```
145 // Función para crear la letra J mayúscula (verde)
146 void CrearLetraJ()
147 {
148     // Vértices para la letra J mayúscula con forma de gancho
149     GLfloat vertices_J[] = {
150         // Barra superior horizontal
151         -0.6f, 0.4f, 0.5f, 0.0f, 1.0f, 0.0f, // 0 - Verde
152         0.6f, 0.4f, 0.5f, 0.0f, 1.0f, 0.0f, // 1
153         0.6f, 0.8f, 0.5f, 0.0f, 1.0f, 0.0f, // 2
154         -0.6f, 0.4f, 0.5f, 0.0f, 1.0f, 0.0f, // 3
155         0.6f, 0.8f, 0.5f, 0.0f, 1.0f, 0.0f, // 4
156         -0.6f, 0.8f, 0.5f, 0.0f, 1.0f, 0.0f, // 5
157         // Barra vertical central
158         -0.2f, -0.8f, 0.5f, 0.0f, 1.0f, 0.0f, // 6
159         0.2f, -0.8f, 0.5f, 0.0f, 1.0f, 0.0f, // 7
160         0.2f, 0.4f, 0.5f, 0.0f, 1.0f, 0.0f, // 8
161         -0.2f, -0.8f, 0.5f, 0.0f, 1.0f, 0.0f, // 9
162         0.2f, 0.4f, 0.5f, 0.0f, 1.0f, 0.0f, // 10
163         -0.2f, 0.4f, 0.5f, 0.0f, 1.0f, 0.0f, // 11
164         // Barra inferior horizontal (gancho)
165         -0.6f, -0.8f, 0.5f, 0.0f, 1.0f, 0.0f, // 12
166         -0.2f, -0.8f, 0.5f, 0.0f, 1.0f, 0.0f, // 13
167         -0.2f, -0.4f, 0.5f, 0.0f, 1.0f, 0.0f, // 14
168         -0.6f, -0.8f, 0.5f, 0.0f, 1.0f, 0.0f, // 15
169         -0.2f, -0.4f, 0.5f, 0.0f, 1.0f, 0.0f, // 16
170         -0.6f, -0.4f, 0.5f, 0.0f, 1.0f, 0.0f, // 17
171         // Barra vertical izquierda inferior (gancho)
172         -0.6f, -0.8f, 0.5f, 0.0f, 1.0f, 0.0f, // 18
173         -0.4f, -0.8f, 0.5f, 0.0f, 1.0f, 0.0f, // 19
174         -0.4f, -0.6f, 0.5f, 0.0f, 1.0f, 0.0f, // 20
175         -0.6f, -0.8f, 0.5f, 0.0f, 1.0f, 0.0f, // 21
176         -0.4f, -0.6f, 0.5f, 0.0f, 1.0f, 0.0f, // 22
177         -0.6f, -0.6f, 0.5f, 0.0f, 1.0f, 0.0f, // 23
178     };
179     MeshColor* letraJ = new MeshColor();
180     letraJ->CreateMeshColor(vertices_J, 144); // 24 vértices * 6 componentes
181     meshColorList.push_back(letraJ);
182 }
```

Shaders para los colores de los cubos y pirámides:

```
195 // Shaders para la casa (usando shader_colores.vert y .frag por color)
196 Shader* shaderRojo = new Shader();
197 shaderRojo->CreateFromFiles("shaders/shader_colores.vert", "shaders/rojo.frag");
198 shaderList.push_back(*shaderRojo); // [2]
199
200 Shader* shaderVerde = new Shader();
201 shaderVerde->CreateFromFiles("shaders/shader_colores.vert", "shaders/verde.frag");
202 shaderList.push_back(*shaderVerde); // [3]
203
204 Shader* shaderAzul = new Shader();
205 shaderAzul->CreateFromFiles("shaders/shader_colores.vert", "shaders/azul.frag");
206 shaderList.push_back(*shaderAzul); // [4]
207
208 Shader* shaderCafe = new Shader();
209 shaderCafe->CreateFromFiles("shaders/shader_colores.vert", "shaders/cafe.frag");
210 shaderList.push_back(*shaderCafe); // [5]
211
212 Shader* shaderVerdeOscuro = new Shader();
213 shaderVerdeOscuro->CreateFromFiles("shaders/shader_colores.vert", "shaders/verdeoscuro.frag");
214 shaderList.push_back(*shaderVerdeOscuro); // [6]
215
```

Sección 1: Iniciales de Colores

```
245 // =====//
246 // SECCIÓN 1: INICIALES DE COLORES
247 // =====//
248
249
250 // Usar shader para figuras con color
251 shaderList[1].useShader();
252 uniformModel = shaderList[1].getModelLocation();
253 uniformProjection = shaderList[1].getProjectLocation();
254
255 // Dibujar la letra D mayúscula (roja) - Margen inferior izquierdo
256 model = glm::mat4(1.0);
257 model = glm::scale(model, glm::vec3(0.8f, 0.8f, 1.0f)); // Escala reducida
258 model = glm::translate(model, glm::vec3(-1.2f, -1.8f, -4.0f));
259 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
260 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
261 meshColorList[0]->RenderMeshColor(); // Letra D
262
263 // Dibujar la letra T mayúscula (azul) - Centro
264 model = glm::mat4(1.0);
265 model = glm::scale(model, glm::vec3(0.8f, 0.8f, 1.0f)); // Misma escala
266 model = glm::translate(model, glm::vec3(0.0f, -0.2f, -4.0f));
267 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
268 meshColorList[1]->RenderMeshColor(); // Letra T
269
270 // Dibujar la letra J mayúscula (verde) - Margen superior derecho
271 model = glm::mat4(1.0);
272 model = glm::scale(model, glm::vec3(0.8f, 0.8f, 1.0f)); // Misma escala
273 model = glm::translate(model, glm::vec3(1.5f, 1.4f, -4.0f));
274 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
275 meshColorList[2]->RenderMeshColor(); // Letra J
276
```

Sección 2: Casa en 3D

```
279 // =====//
280 //                               SECCIÓN 2: CASA EN 3D
281 // =====//
282
283 // Actualizar ángulo para rotación (más lento)
284 angulo += 0.025f; // Reducido de 0.5f a 0.025f para que gire más lento
285
286 // CASA: CUERPO DE LA CASA (cubo rojo)
287 shaderList[2].useShader(); // Shader rojo (índice 2)
288 uniformModel = shaderList[2].getModelLocation();
289 uniformProjection = shaderList[2].getProjectLocation();
290
291 model = glm::mat4(1.0);
292 model = glm::scale(model, glm::vec3(2.0f, 2.0f, 2.0f));
293 model = glm::translate(model, glm::vec3(0.0f, 0.0f, -4.0f));
294 model = glm::rotate(model, glm::radians(angulo), glm::vec3(0.0f, 1.0f, 0.0f));
295 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
296 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
297 meshList[1]-->RenderMesh(); // Cubo (índice 1 en meshList)
298
299
300 // CASA: TECHO AZUL (pirámide)
301 shaderList[4].useShader(); // Shader azul (índice 4)
302 uniformModel = shaderList[4].getModelLocation();
303 uniformProjection = shaderList[4].getProjectLocation();
304 model = glm::mat4(1.0);
305
306 // Escala: más ancha que la casa
307 model = glm::scale(model, glm::vec3(2.8f, 1.5f, 2.5f));
308 // Posición: encima del cubo
309 model = glm::translate(model, glm::vec3(0.0f, 1.25f, -4.0f));
310 // Misma rotación que el cubo
311 model = glm::rotate(model, glm::radians(angulo), glm::vec3(0.0f, 1.0f, 0.0f));
312 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
313 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
314 meshList[0]-->RenderMesh(); // Pirámide (índice 0 en meshList)
315
316 // CASA: PUERTA VERDE
317 shaderList[3].useShader(); // Shader verde (índice 3)
318 uniformModel = shaderList[3].getModelLocation();
319 uniformProjection = shaderList[3].getProjectLocation();
320 model = glm::mat4(1.0);
321 // Escala: ancho 0.05f, alto 0.05f, profundo 0.2f
322 model = glm::scale(model, glm::vec3(0.05f, 0.05f, 0.2f));
323 // Posición: centrado y hacia abajo
324 model = glm::translate(model, glm::vec3(0.0f, -1.0f, -3.9f));
325 // Misma rotación que la casa
326 model = glm::rotate(model, glm::radians(angulo), glm::vec3(0.0f, 1.0f, 0.0f));
327 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
328 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
329 meshList[1]-->RenderMesh(); // Cubo (índice 1 en meshList)
```



```

330
331 // CASA: VENTANA DERECHA VERDE
332 shaderList[3].useShader(); // Shader verde (índice 3)
333 uniformModel = shaderList[3].getModelLocation();
334 uniformProjection = shaderList[3].getProjectLocation();
335 model = glm::mat4(1.0);
336 // Escala: ancho 0.04f, alto 0.04f, profundo 0.2f
337 model = glm::scale(model, glm::vec3(0.04f, 0.04f, 0.2f));
338 // Posición: arriba y hacia la derecha
339 model = glm::translate(model, glm::vec3(1.5f, 1.0f, -3.9f));
340 // Misma rotación que la casa
341 model = glm::rotate(model, glm::radians(angulo), glm::vec3(0.0f, 1.0f, 0.0f));
342 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
343 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
344 meshList[1]->RenderMesh(); // Cubo (índice 1 en meshList)
345
346
347 // CASA: VENTANA IZQUIERDA VERDE
348 shaderList[3].useShader(); // Shader verde (índice 3)
349 uniformModel = shaderList[3].getModelLocation();
350 uniformProjection = shaderList[3].getProjectLocation();
351 model = glm::mat4(1.0);
352 // Escala: ancho 0.04f, alto 0.04f, profundo 0.2f
353 model = glm::scale(model, glm::vec3(0.04f, 0.04f, 0.2f));
354 // Posición: arriba y hacia la izquierda
355 model = glm::translate(model, glm::vec3(-1.5f, 1.0f, -3.9f));
356 // Misma rotación que la casa
357 model = glm::rotate(model, glm::radians(angulo), glm::vec3(0.0f, 1.0f, 0.0f));
358
359 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
360 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
361 meshList[1]->RenderMesh(); // Cubo (índice 1 en meshList)
362
363 // CASA: TRONCO DERECHO (café)
364 shaderList[5].useShader(); // Shader café (índice 5)
365 uniformModel = shaderList[5].getModelLocation();
366 uniformProjection = shaderList[5].getProjectLocation();
367 model = glm::mat4(1.0);
368 // Escala: ancho 0.04f, alto 0.04f, profundo 0.2f (forma de tronco)
369 model = glm::scale(model, glm::vec3(0.04f, 0.04f, 0.2f));
370 // Posición: a la derecha de la casa (X positivo), en la base (Y negativo)
371 model = glm::translate(model, glm::vec3(7.5f, -2.6f, -4.5f));
372 // Misma rotación que la casa
373 model = glm::rotate(model, glm::radians(angulo), glm::vec3(0.0f, 1.0f, 0.0f));
374 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
375 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
376 meshList[1]->RenderMesh(); // Cubo (índice 1 en meshList)
377
378 // CASA: TRONCO IZQUIERDO (café)
379 shaderList[5].useShader(); // Shader café (índice 5)
380 uniformModel = shaderList[5].getModelLocation();
381 uniformProjection = shaderList[5].getProjectLocation();
382 model = glm::mat4(1.0);
383 // Escala: ancho 0.04f, alto 0.04f, profundo 0.2f (forma de tronco)
384 model = glm::scale(model, glm::vec3(0.04f, 0.04f, 0.2f));
385 // Posición: a la derecha de la casa (X negativo), en la base (Y negativo)
386 model = glm::translate(model, glm::vec3(-7.5f, -2.6f, -4.5f));
387 // Misma rotación que la casa
388 model = glm::rotate(model, glm::radians(angulo), glm::vec3(0.0f, 1.0f, 0.0f));

```

```

388 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
389 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
390 meshList[1]->RenderMesh(); // Cubo (índice 1 en meshList)
391
392 // CASA: ÁRBOL DERECHO (verde oscuro)
393 shaderList[6].useShader(); // Shader verde oscuro (índice 6)
394 uniformModel = shaderList[6].getModelLocation();
395 uniformProjection = shaderList[6].getProjectLocation();
396 model = glm::mat4(1.0);
397 // Escala: ancho 0.5f, alto 0.8f, profundo 0.8f (forma de árbol)
398 model = glm::scale(model, glm::vec3(0.5f, 0.8f, 0.8f));
399 // Posición: sobre el tronco derecho (X positivo), en la base del tronco
400 model = glm::translate(model, glm::vec3(2.15f, 0.13f, -4.0f));
401 // Misma rotación que la casa
402 model = glm::rotate(model, glm::radians(angulo), glm::vec3(0.0f, 1.0f, 0.0f));
403 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
404 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
405 meshList[0]->RenderMesh(); // Pirámide (índice 0 en meshList)
406
407
408 // CASA: ÁRBOL IZQUIERDO (verde oscuro)
409 shaderList[6].useShader(); // Shader verde oscuro (índice 6)
410 uniformModel = shaderList[6].getModelLocation();
411 uniformProjection = shaderList[6].getProjectLocation();
412 model = glm::mat4(1.0);
413 // Escala: ancho 0.5f, alto 0.8f, profundo 0.8f (forma de árbol)
414 model = glm::scale(model, glm::vec3(0.5f, 0.8f, 0.8f));
415 // Posición: sobre el tronco derecho (X negativo), en la base del tronco
416 model = glm::translate(model, glm::vec3(-2.15f, 0.13f, -4.0f));
417 // Misma rotación que la casa
418 model = glm::rotate(model, glm::radians(angulo), glm::vec3(0.0f, 1.0f, 0.0f));
419 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
420 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
421 meshList[0]->RenderMesh(); // Pirámide (índice 0 en meshList)
422

```

Shaders creados:

shader_colores.vert:

```
shader_colores.vert
Archivo Editar Ver

#version 330
layout (location =0) in vec3 pos;
uniform mat4 model;
uniform mat4 projection;

void main()
{
    gl_Position = projection * model * vec4(pos, 1.0f);
}
```

Shader de color rojo (rojo.frag):

```
rojo.frag
Archivo Editar Ver

#version 330
out vec4 color;

void main()
{
    color = vec4(1.0f, 0.0f, 0.0f, 1.0f); // Rojo
}
```

Shader de color verde (verde.frag):

```
verde.frag
Archivo Editar Ver

#version 330
out vec4 color;

void main()
{
    color = vec4(0.0f, 1.0f, 0.0f, 1.0f); // Verde
}
```

Shader de color azul (azul.frag):

```
azul.frag x +
Archivo Editar Ver

#version 330
out vec4 color;

void main()
{
    color = vec4(0.0f, 0.0f, 1.0f, 1.0f); // Azul
}
```

Shader de color café (cafe.frag):

```
cafe.frag x blanco.frag +
Archivo Editar Ver

#version 330
out vec4 color;

void main()
{
    color = vec4(0.478f, 0.255f, 0.067f, 1.0f); // Café
}
```

Shader de color verde oscuro (verdeoscuro.frag):

```
verdeoscuro.frag x +
Archivo Editar Ver

#version 330
out vec4 color;

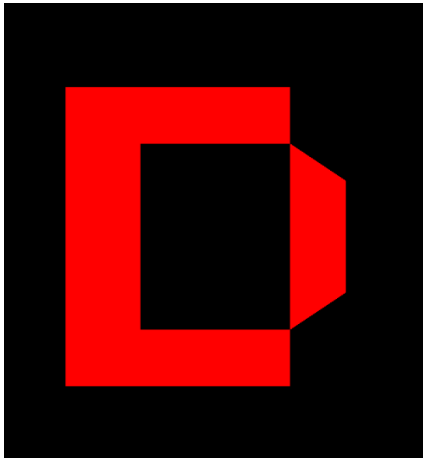
void main()
{
    color = vec4(0.0f, 0.5f, 0.0f, 1.0f); // Verde oscuro
}
```

2.- Liste los problemas que tuvo a la hora de hacer estos ejercicios y si los resolvió explicar cómo fue, en caso de error adjuntar captura de pantalla

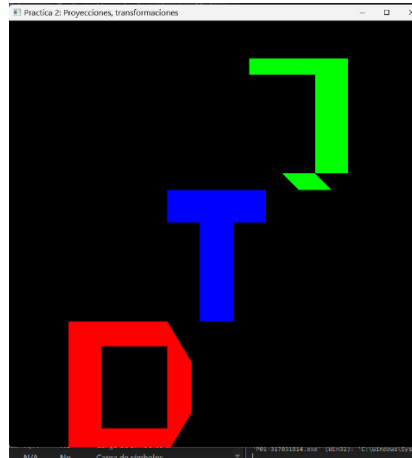
Actividad 1 errores/problemas:

El principal problema de la actividad 1 consistió en necesitar volver a elaborar las letras de mi nombre ya que las empleadas en la ocasión pasada causaban problemas de diseño y eran muy difíciles de manejar

Imágenes de los errores:



Error 1 (D con errores en la curva).

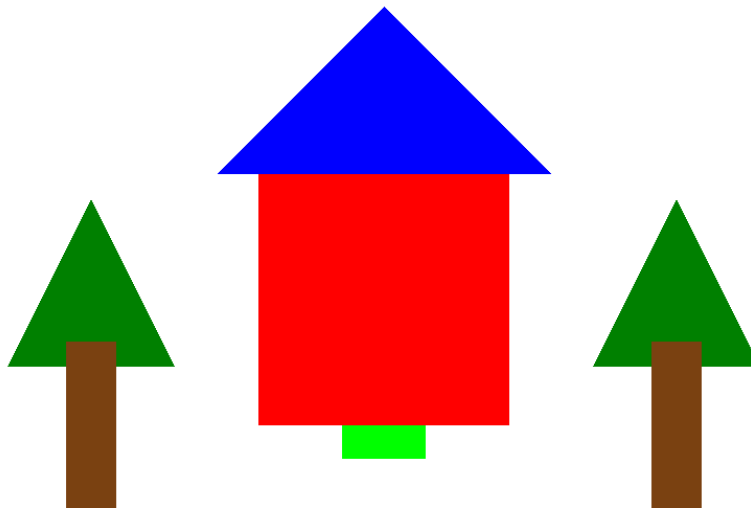


Error 2 (la J se distorsionó en proporciones).

Actividad 2 errores/problema:

El principal problema de la actividad 2 fue que la figura de la pirámide no permitía hacer un techo que me agradase, y aunque técnicamente intenté con 4 pirámides para hacerlo el resultado no me gustaba así que reestructure los vértices y opté por emplear una pirámide cuadrangular para poder hacer los techos y árboles de forma que se apreciaran mejor

Otro problema fue que al detener las rotaciones e ir usando la traslación y escalados correspondientes en ocasiones sufría distorsiones graciosas como que las ventanas y puerta quedaran dentro del cubo o se alargaran demasiado los cubos, además cuando rotaba con fondo blanco tenía errores en el movimiento así que opté por quitar el fondo blanco y otro elemento a destacar fue que necesite reducir la velocidad de rotación para poder apreciar el efecto 3D con mayor detalle pasando de 0.5 a 0.025



Error 3 (Elementos distorsionados en profundidad y largo)

3.- Conclusión:

- a. Los ejercicios del reporte: Complejidad, Explicación.

Referente a la complejidad de los ejercicios en esta ocasión tener que rehacer todas mis letras fue una problemática inesperada además de que el manejar campos en 3 dimensiones en un inicio parece fácil, pero me tope con situaciones donde manejaba valores “correctos” y sin embargo tenía problemáticas para poder acomodar de forma correcta los elementos

- b. Comentarios generales: Faltó explicar a detalle, ir más lento en alguna explicación, otros comentarios y sugerencias para mejorar desarrollo de la práctica

Mi comentario general respecto a la sesión de laboratorio es que debido al limitado tiempo sentí que en esta ocasión la teoría a pesar de ser vista fue tanta en cantidad que me terminé saturando y confundiendo al momento de seguir las explicaciones, no obstante y por paradójico que suene me hubiera gustado un poco más de tiempo

a ver el funcionamiento del escalado y traslación en el ejemplo visto en la sesión ya que sentí que se vio muy rápido

c. Conclusión

En líneas generales, esta práctica me ayudó a comprender la transición del espacio 2D al espacio tridimensional y la importancia de los shaders personalizados para controlar el color de los objetos. Aprendí que, a diferencia de la práctica anterior donde el color viajaba con los vértices, aquí cada elemento requiere su propio shader, lo que permite un control mayor fino sobre la apariencia de los objetos.

Descubrí que aplicar la rotación antes o después de la traslación produce efectos radicalmente diferentes (órbita vs rotación local), y que funciones como `glm::translate`, `glm::rotate` y `glm::scale` es esencial para posicionar correctamente cada elemento. También aprendí a modificar la geometría base, como reestructurar los vértices de la pirámide triangular a una pirámide cuadrangular para lograr un techo más estético.

Al final, me queda claro que, aunque el código pueda parecer extenso para construir una casa con árboles, cada línea representa el control total sobre el proceso gráfico: desde la definición de vértices hasta la animación continua que hace girar toda la escena para poder visualizar el efecto 3D

Bibliografía

- *LearnOpenGL - Advanced GLSL*. (n.d.). <https://learnopengl.com/Advanced-OpenGL/Advanced-GLSL>
- *LearnOpenGL - Coordinate Systems*. (n.d.). <https://learnopengl.com/Getting-started/Coordinate-Systems>
- *CSSE351 Computer Graphics*. (n.d.). <https://www.rose-hulman.edu/class/csse/csse351-old/m3/Using%20transforms/>
- *Stevewhims*. (n.d.). *Función glViewport (GL.h) - Win32 apps*. Microsoft Learn. <https://learn.microsoft.com/es-mx/windows/win32/opengl/glviewport>